If you look at the request body closely, the `properties` object is defined at the root level. This is unlike the earlier method of creating a schema using the indexing API, where the `properties` object was wrapped in the root-level `mappings` object.

UPDATING AN EMPTY INDEX

We can also use the same principle of updating the schema on an empty index. An *empty index* is created without schema mapping (for example, executing the `PUT books` command creates an empty `books` index).

Similar to how we updated the index in listing 4.5 by calling the `_mapping` endpoint with the required schema definition, we can use the same approach for the empty index. The following snippet updates the schema on the `departments` index with an additional field.

Listing 4.6 Updating the mapping schema of an empty index

```
PUT departments                          <-- Creates an empty index first

PUT departments/_mapping                 <-- Uses the mapping API to update an empty index
{
  "properties":{
    "name":{
      "type":"text"                      <-- The name field is declared as a text type.
    }
  }
}
```

So far, we've seen the additive case of updating the schema with additional fields. But what if we want to change the data types of the existing fields? Suppose we want to change a field's type from `text` to `keyword`. Can we update the mapping just as easily as we added fields? The short answer is no. Elasticsearch doesn't allow us to change or modify the data types of existing fields. Instead we have to do a bit more work, as discussed in the next section.

4.3.3 Modifying existing fields is not allowed

Once an index is live (created with data fields and operational), any modifications of existing fields on the live index are prohibited. For example, if a field was defined as a `keyword` data type and indexed, it cannot be changed to a different data type (say, from `keyword` to `text`). There is a good reason for this. Data is indexed using existing schema definitions and, hence, stored in the index. Searching on that field data fails if the data type has been modified, which leads to an erroneous search experience. To avoid search failures, Elasticsearch does not allow us to modify existing fields.

Well, you may ask, what is the alternative? Business requirements change, as do technical requirements. How can we fix data types (which may have been created incorrectly) on a live index?

We can use *reindexing*. Reindexing operations source data from the original index to a new index (which can have updated schema definitions). The idea is as follows: